

Título do trabalho de conclusão de curso (o título deve ser curto, claro, afirmativo e conclusivo. Deve conter no máximo 15 palavras e não deve conter expressões redundantes como: “Estudo de...”; “Influência de...”; “Elaboração de...” “Efeito de...” “Análise de...”)

nome completo aluno^{1*}; nome completo orientador²

¹ Nome da Empresa ou Instituição (opcional). Titulação ou função ou departamento. Endereço completo (pessoal ou profissional) – Bairro; 00000-000 Cidade, Estado, País

² Nome da Empresa ou Instituição (opcional). Titulação ou função ou departamento. Endereço completo (pessoal ou profissional) – Bairro; 00000-000 Cidade, Estado, País

*autor correspondente: nome@email.com

Título do trabalho de conclusão de curso

Resumo (ou Sumário Executivo)

Tópico obrigatório para o depósito do TCC, porém opcional para a etapa dos Resultados Preliminares. Para o curso de Gestão de Projetos, devido à sua particularidade, o Resumo pode ser substituído pelo **Sumário Executivo**.

Palavras-chave: (inserir até cinco palavras diferentes das contidas no título, separadas por ponto-e-vírgula).

Atenção: antes de enviar o arquivo para o [Sistema de TCCs](#), remova todas as instruções originais que estão abaixo do conteúdo dos tópicos.

Introdução

No contexto de programação orientada a objetos, as abstrações são fundamentais para organização e atribuição de responsabilidades. A compreensão da lógica é simples quando os assuntos são comumente conhecidos e/ou com poucos níveis de abstrações. Entretanto, o código se torna mais complexo e, por consequência, demanda mais esforço e tempo para compreendê-lo quando se trata de uma base de código extensa, desenvolvida durante vários anos, por vários profissionais e com foco em um conhecimento específico. Em uma ótica empresarial, este cenário implica em maiores investimentos para que novos engenheiros sejam incorporados à equipe por causa da complexidade, e adiciona um nível de abstração extra nas tomadas de decisão relacionadas ao desenvolvimento do *software* por não ter uma visão geral do todo. Sob essa perspectiva, visualizar a organização de uma base de código clara e intuitivamente agrega valor, principalmente, na rotina das empresas desenvolvedoras de *softwares*.

A visualização e organização de informações de um *software* auxilia no seu gerenciamento, o que, de forma secundária, reflete na maior manutenibilidade do produto. Martin (2009) ilustra essa dinâmica ao comparar produtividade pelo tempo de existência do *software*: ao passo que a extensão e complexidade do código aumentam, especialmente sem planejamento, as chances de se transformar em um emaranhado de soluções improvisadas também aumentam. Nesse cenário, o autor argumenta que a produtividade para desenvolver novas funcionalidades e a manutenção do código declina drasticamente a médio e longo prazo, quando não gerenciado corretamente, tornando-se quase impraticável e economicamente inviável.

O ponto de partida para projetos dessa natureza contempla a análise de código fonte, tema de investigação consolidado para diversas áreas na computação: a otimização de compilador, abordado por Dean (1995), que expõe os benefícios ao realizar análises de hierarquia de classes como método para resolver ambiguidades em invocações de funções

virtuais; análises de qualidade de *software*, abordado por Ceconello (2016), apresenta alguns dos problemas recorrentes na programação orientada a objetos; e estudos até mesmo no ramo da avaliação do quão eficiente estão os modelos de inteligência artificial baseados em linguagens (LLMs) quando comparado programação funcional com orientada a objetos, abordado por Wang (2024), que evidencia a necessidade de melhorias.

Inúmeras aplicabilidades, soluções e produtos foram propostos com objetivo de aprimorar a experiência durante e após o desenvolvimento de *software*. Silva (2023) detalha os benefícios gerais das ferramentas que podem ser integradas em diversos momentos no ciclo de vida do *software*, desde a etapa de codificação até a validação do produto final. Como exemplos temos: o SonarQube, uma plataforma de análise de código com foco em qualidade e cobertura de testes; o *Clang Static Analyzer*, uma ferramenta de análise nativa do LLVM aplicada ao C e C++; e *AddressSanitizer*, como ferramenta para identificar erros de memória em tempo de execução.

Frente ao exposto, ferramentas de visualização e organização de uma base de código são bastante requisitadas em projetos de *software*. Por consequência, é tema de pesquisas tanto no âmbito acadêmico quanto no desenvolvimento de novos produtos e soluções no setor privado. A literatura relacionada a este projeto de pesquisa está associada com a investigação feita em nível de compiladores e nos ambientes de desenvolvimento integrado (IDEs), com o intuito de auxiliar o programador na escrita e otimização da compilação de código. Entretanto, o mapeamento estruturado para facilitar discussões e a tomada de decisão, não tem sido amplamente abordado e/ou mencionados em relevantes estudos e produtos. Sendo assim, se faz relevante arquitetar um software capaz de realizar uma análise estática em códigos baseados em programação orientada a objetos, extrair informação sobre a organização das classes e expor, de maneira simples e direta, informações estruturadas cerca do código em questão.

Metodologia ou Material e Métodos

O presente trabalho fundamenta-se na análise de código que, segundo a literatura, pode ser realizada tanto por uma abordagem estática quanto dinâmica. Um comparativo é apresentado por Silva (2023), que ressalta os aspectos positivos e os reveses de cada estratégia. O autor enfatiza o potencial positivo alcançado quando utilizadas em conjunto. No contexto do projeto de pesquisa proposto, a análise estática revela-se uma opção promissora por depender unicamente do código, sem a necessidade de compilá-lo, condição que torna o projeto proposto agnóstico em relação à linguagem de programação e

permite avaliar a intenção original do desenvolvedor ao examinar o código antes de quaisquer otimizações do compilador.

As etapas iniciais do projeto tiveram como foco a implementação de compiladores, com início no mapeamento do código fonte e evoluíram para a organização das informações em forma de tokens. Essa categoria de dados constitui a entrada para o processo de *parsing*, responsável por agrupar os tokens em expressões sintáticas mais abrangentes e viabilizar a construção das chamadas árvores de sintaxe. Essas, por sua vez, viabilizam o processo de análise do código de acordo com a especificidade de cada linguagem de programação. As informações necessárias para o projeto de pesquisa proposto foram coletadas nesta etapa e dizem respeito, principalmente, à árvore de sintaxe.

Embora existam outras etapas em um projeto de compilador, o resultado produzido por elas não agrega informações úteis ao presente projeto. Ademais, Nystrom (2020) apresenta uma visão detalhada de todo o processo, explicando a criação de um interpretador, incluindo as etapas: otimização do código fonte; tradução para linguagem de baixo nível mais próxima de máquina e compilação conforme a escolha do ambiente de execução do *software*.

Os autores Nystrom (2020), Aho (2007), e Cooper (2012) foram as referências principais para fundamentar o tema compiladores. Detalhando as partes importantes para o presente trabalho, temos o mapeamento dos caracteres de um código como a primeira das três etapas. Nesta fase inicial, a performance é crucial, sendo o momento de maior demanda computacional por requerer uma análise sequencial de todos os caracteres dos arquivos de entrada do sistema. Todas as informações precisam ser categorizadas para possibilitar o agrupamento dos caracteres em *tokens*, que constituem uma abstração de informações úteis para o interpretador. Em termos simplificados, é o menor agrupamento de caracteres com representabilidade em um código, incluindo o tratamento de ambiguidades. A identificação dos *tokens* é realizada, dentre outras maneiras, por meio das expressões regulares, capazes de identificar padrões definidos na gramática léxica da linguagem de programação utilizada no código analisado.

A segunda fase é o *parser*, que determina se a sequência de *tokens*, gerada pelo mapeamento do código, constitui uma sentença válida e o insere em uma representação que emprega árvores de sintaxe para organizar as expressões lógicas. Obtém-se, como resultado intermediário, a tradução do código para uma linguagem livre de contextos que simplifica a interpretação do token utilizando de modelos de algoritmos já validados como: *top-down* e *bottom-up*. O *top-down* correlaciona as entradas dos *tokens* com as produções gramaticais, visando a eficiência ao prever a próxima palavra. O *bottom-up* opera com os

detalhes, a precisa sequência de palavras é mantida em memória até que identificado o respectivo contexto.

Por fim, a fase de análise de contexto, no âmbito dos compiladores, é responsável por realizar um processamento adicional ao *parser*, identificando erros funcionais uma vez que o código está organizado de forma estruturada na árvore de sintaxe. No contexto do projeto de pesquisa proposto, essa etapa é explorada de maneira limitada, restringindo a identificação da relação entre classes em um código.

Frente ao contexto apresentado e o panorama funcional do *software*, é pertinente discutir como ele está implementado. Em uma visão de alto nível, considerando o *software* uma caixa preta, o mesmo recebe uma base de código como entrada e retorna uma estrutura de dados no formato *.json* para utilização posterior. Já em uma visão detalhada do processo, é possível suportar várias linguagens de programação, mapear diversos tipos de *tokens* para cada uma das linguagens e performar diversos tipos de operações dependendo do *token* e da linguagem selecionada.

O escopo do projeto de pesquisa restringe-se a códigos com linguagem C++, em projetos listados no *gitHub* e avaliando somente estruturas de classes. Entretanto, a escalabilidade para suportar outros tipos de linguagens e análises é considerada na arquitetura do software, que contempla possíveis expansões sem a necessidade de refatorações significativas.

Na discussão da implementação da pesquisa, o padrão de projeto *visitor* é um exemplo que aborda o antagonismo entre inserção de tipos e operações em uma lógica. Iglberger (2022) discorre sobre o tema com análises sobre a escolha do paradigma de programação frente a necessidade de adicionar tipos ou operações. Ao passo que em uma programação procedural seja simples adicionar operações novas, adicionar tipos tem por consequência alterar componentes base do código e propagar a mudança para as demais camadas. Em contrapartida, a programação orientada a objetos simplifica a adição de novos tipos mas complica a inserção de novas operações. O padrão *visitor* atua, junto a orientação a objetos, para simplificar a inserção de operações ao delegar a implementação da operação para uma classe externa que será, em termos simples, aceita pela classe principal.

O *visitor* convencional transforma o estilo de programação orientado a objetos de simples inserções de tipos para difícil e de difícil inserção de operação para simples, mas existe uma alternativa que simplifica ambas inclusões, o *std::variant* do C++. Tal alternativa reserva em memória o espaço necessário para o maior tipo que o *visitor* pode operar e utiliza desse bloco de memória para definir a ação e a classe que será executada. A

viabilidade da utilização do *std::variant* foi avaliada e, diante de seu potencial, está prevista sua incorporação no projeto final.

O padrão *visitor* também é abordado por Nystrom (2020) no contexto do *parser*, ressaltando a simplicidade de adicionar novas operações em um conjunto de tipos específicos sem alterar a implementação da classe em questão. Outros padrões de projeto podem e devem ser utilizados, embora a identificação de todos os contextos e detalhes de implementação que justifiquem sua escolha ainda estejam em análise. À medida que validado os benefícios do padrão para o projeto e com base nas explicações fornecidas por Iglberger (2022), serão implementados.

Das estratégias de implementação para o ferramental a ser utilizado, a linguagem C++ é utilizada pela performance e eficiência. A estrutura do *software* em formato de uma API RESTful segue a tendência em proporcionar praticidade na utilização, sem a necessidade de transferir e instalar localmente, simplesmente interagir com o serviço e obter a informação desejada. Resultado disponibilizado em formato .json, amplamente adotado em APIs e de simples conversão para diversas aplicações. A biblioteca em C++ utilizada para implementar a API é a *Crow*, e a escolha baseia-se no suporte ativo da ferramenta no GitHub e na presença de uma documentação elaborada.

Frente ao projeto proposto, foi fundamental associar a implementação de testes ao desenvolvimento do projeto para assegurar a qualidade e confiabilidade do *software*. Arelado ao projeto de pesquisa foi pertinente desenvolver dois tipos de testes: unitários e de integração. Os testes unitários foram utilizados na validação de componentes da implementação, de forma isolada, explorando as particularidades e detalhes das lógicas implementadas. O *framework* escolhido para os testes unitários é o *Google Tests*, amplamente adotado na indústria e com fácil acesso a documentação. A metodologia de implementação dos testes segue o padrão do *Test-Driven Development (TDD)* abordado por Langr (2013), que ilustra os conceitos utilizando o *framework* com C++. Embora fundamentais no desenvolvimento, os testes unitários não são capazes de abranger a completude do algoritmo. Por isso, foram complementados por testes de integração para, em uma visão de alto nível, avaliar se o *software* se comporta da maneira projetada. Este modelo de teste consiste em prover entradas já conhecidas ao *software* e avaliar o retorno obtido com o esperado para aquela determinada entrada. Essa combinação estratégica de testes explorou a granularidade dos testes unitários com a abrangência dos testes de integração para garantir um *software* de melhor qualidade.

Frente a amplitude dos temas abordados, a criação de um repositório para orquestrar todo o desenvolvimento do projeto mostrou-se essencial. Vislumbra-se também a criação de uma rotina de integração constante (CI), para compilar e executar testes sempre que alguma

alteração for inserida no repositório. Em um horizonte futuro, também vislumbra a implementação do despacho contínuo (CD) de atualização sempre que houver uma versão estável. Contudo, este foi um assunto pouco explorado e que requer investigação com mais detalhes para definição de como será realizada sua implementação. Ainda existem questões sem definições precisas mas que já foram mapeadas para implementação, como pontos de melhoria não críticos ao produto final, a infraestrutura da hospedagem da API, automatização da compilação em cada mudança no repositório e acionamento da execução dos testes (unitários e de integração) após cada compilação.

Resultados Preliminares

O título da seção Resultados Preliminares deve ser alinhado à esquerda, grafado em negrito com as primeiras letras das palavras em letras maiúsculas. É permitido que a seção seja dividida em subtópicos, seguindo a de acordo com a descrição feita no item 16.5 Resultados e Discussão e apresentados na mesma ordem da seção Material e Métodos. Nesta seção devem ser apresentados os resultados parciais obtidos na pesquisa, ou seja, os resultados obtidos até o momento.

Atenção: antes de enviar o arquivo para o Sistema de TCCs, remova todas as instruções originais que estão abaixo do conteúdo dos tópicos.

- Apresentação da visão do todo do projeto. Explicando o que vai ser focado e o que não vai ser detalhado
- Explicar o que está sendo feito em cada bloco (deixar o backend por último)
 - escrever estrutura da api (rodando localmente)
 - Descrever a estrutura de testes, (coocar imagens?)
 - Detalhar o que foi feito no backend
 - Infraestrutura de logs para a aplicação
 - Implementação da comunicação com github
 - Download dos arquivos
 - Estrutura de grafos para salvar info das pastas
 - Leitura dos arquivos
 -
-

Conclusão(ões) ou Considerações Finais

Tópico obrigatório para o depósito do TCC, porém opcional para a etapa dos Resultados preliminares.

Atenção: antes de enviar o arquivo para o Sistema de TCCs, remova todas as instruções originais que estão abaixo do conteúdo dos tópicos.

Referências

Aho,A.V; Lam,M.S; Sethi, R; Ullman, J.D. 2007. Compilers: Principles, Techniques, & Tools. Segunda Edição. Pearson Education, Inc, Boston, Massachusetts, Estados Unidos.

Ceconello, J. 2016. Ferramenta de auxílio à análise de anomalias em códigos Orientados a Objeto. Monografia em Ciência da Computação. Universidade Federal do Rio Grande do Sul, Porto Alegre, Rio Grande do Sul, Brasil.

Cooper, K.D; Torczon,L. 2012. Engineering a Compiler. Segunda Edição. ELSEVIER, Huston, Texas, Estados Unidos.

Dean,J; Grove,D; Chambers,C. 1995. Optimization of Object-Oriented Programs Using Static Class Hierarchy Analysis. 9th European Conference: 77-101.

Iglberger, K. 2022. C++ Software Design: Design Principles and Patterns for High-Quality Software. O'Reilly Media, Inc, Sebastopol, Califórnia, Estados Unidos.

Langr, J. 2013. Modern C++ Programming with Test-Driven Development. The Pragmatic Programmers, LLC. Colorado Springs, Colorado, Estados Unidos.

Martin, R.C. 2009. Clean Code A Handbook of Agile Software Craftsmanship. Pearson Education, Inc, Boston, Massachusetts, Estados Unidos.

Nystrom, R. 2015-2020. Crafting Interpreters. Genever Benning. Seattle, Washington, Estados Unidos.

Silva, D; Samarasekara, H.M.P.P.K.H; Hettiarachchi, R.T; Senanayake, W.A.B.M; Sanjaya, A.M.T; Abeysekara, M.P. 2023. A Comparative Analysis of Static and Dynamic Code Analysis Techniques. TechRxiv: DOI: 10.36227/techrxiv.22810664.v1.

Wang, S; Ding, L; Shen, L; Luo, Y; Du, B; Tao, D. 2024. Título do trabalho. Findings of the Association for Computational Linguistics: 13619–13639.